Notebook 03: **Proxy Variant Builder and Signal Behavior Analysis**

Purpose of This Notebook

This notebook is the first true signal-analysis extension in the final pipeline.

Notebook 01 established the baseline story:

- the data pipeline works,
- a simple range-aware proxy $\hat{\rho} = I \cdot R$ is meaningful,
- the gains are modest and scene-dependent.

Notebook 02 established the temporal structure:

- multiple contiguous windows across sequence 00,
- short, medium, and long scales,
- fully validated frame and label alignment,
- reproducible window metadata.

This notebook now moves to the next critical question:

> If $I \cdot R$ works, is it actually the best practical proxy?

Why This Notebook Exists

The preliminary stage treated the proxy:

$$\hat{\rho}_i = I_i \cdot R_i$$

as a working idea, not an optimized design.

That was correct.

But for the final project, **proxy formulation must be treated as an empirical problem**, not a fixed assumption.

Different transformations of intensity and range can:

- reshape the signal distribution,
- alter range dependence,
- amplify or suppress noise,
- affect numerical stability,
- and ultimately influence semantic usefulness.

So before doing large-scale multi-window analysis, we must first answer:

Which proxy formulations are numerically stable, interpretable, and worth carrying forward?

Scientific Motivation

From the LiDAR signal model:

$$I_i \propto \frac{\rho_i \cos(\alpha_i)}{R_i^2}$$

raw intensity is entangled with:

- reflectivity ρ_i,
- incidence angle α_i,
- and range R_i.

The proxy:

$$\hat{\rho}_i = I_i \cdot R_i$$

partially compensates for range, but it is:

- not physically calibrated,
- still range-dependent,
- and purely heuristic.

So we generalize the idea:

$$\phi(I, R) = \text{range-aware transformation of intensity}$$

and study a family of candidate proxies.

Candidate Proxy Family

We define a structured set of proxy variants:

Baseline

$$\phi_0(I, R) = I$$

Core Variants

$$\phi_1(I, R) = I \cdot R$$

$$\phi_2(I, R) = I \cdot R^2$$

Log-Scaled Variant

$$\phi_3(I, R) = \log(1 + I \cdot R)$$

Clipped Variant

$$\phi_4(I, R) = \text{clip}(I \cdot R)$$

Normalized Variants

$$\phi_5(I, R) = \text{percentile-normalized}(I \cdot R)$$

$$\phi_6(I, R) = \text{robust-scaled}(I \cdot R)$$

Additional normalization regimes:

- per-frame normalization,
- per-window normalization.

Core Questions This Notebook Answers

This notebook does not attempt semantic conclusions yet.

Instead, it focuses on signal-level diagnostics:

1. Distribution Behavior

- How does each proxy reshape the signal distribution?
- Does it introduce heavy tails or collapse structure?

2. Range Dependence

- How strongly is each proxy correlated with range?
- Does it overcompensate (e.g., $I \cdot R^2$)?

3. Numerical Stability

- Does the proxy remain bounded?
- Does it explode under large range values?

4. Interpretability

- Does the signal remain structured and meaningful?
- Does it retain usable dynamic range?

Mathematical Diagnostics

For each proxy ϕ_k, we compute:

Distribution Summary

$$\{\min, p25, p50, p75, p95, \text{mean}, \text{std}\}$$

Range Correlation

$$\mathrm{corr}\left(\phi_k, R\right)$$

Stability Indicators

- upper-tail growth,
- variance magnitude,
- sensitivity to large R.

Expected Observations (Based on Preliminary Work)

From Notebook 01:

- $I \cdot R$ showed:
  - improved structure,
  - moderate scaling,
  - acceptable stability.
- $I \cdot R^2$ showed:
  - aggressive scaling,
  - heavy-tail explosion,
  - strong range dominance.

So we expect:

- some variants will be clearly **too aggressive**,
- some will be **too weak (close to raw intensity)**,
- and a few will form a **balanced middle ground**.

What This Notebook Will Do

Step-by-step:

1. Load a representative window (not full sequence yet)
2. Compute base signals:
   - intensity I.
   - range R.
3. Construct proxy variants $\phi_k\left(I, R\right)$.
4. Compare numerical distributions
5. Measure correlation with range
6. Visualize distributions
7. Identify unstable or overcompensating proxies
8. Build a compact comparison table
9. Shortlist promising candidates

What This Notebook Is Not Allowed to Do

This is critical.

This notebook is **not** the place to:

- claim semantic improvement,
- perform class-wise analysis,
- compare across all windows,
- compute temporal consistency,
- identify failure cases,
- generate videos or GIFs,
- claim any proxy is universally better.

That belongs in later notebooks.

This notebook is strictly a **signal-level proxy design and screening stage**.

Expected Outputs

By the end of this notebook, we should have:

- a defined set of proxy variants,
- numerical summaries for each proxy,
- distribution plots,
- range correlation comparisons,
- identification of unstable / overcompensating variants,
- a shortlist of promising proxies for further study.

Core Honesty Statement

This notebook does not prove that any proxy is superior for semantic tasks.

It only evaluates:

- numerical behavior,
- stability,
- and interpretability.

All proxies remain heuristic transformations, not calibrated reflectivity estimates.

Final Takeaway of This Notebook

If this notebook succeeds, we will have:

> a small, well-justified set of proxy candidates that are numerically stable and worth evaluating across multiple windows in the next stage.

Not the final answer.

But the correct candidates.

And that’s exactly what this stage is supposed to do.

Load a Single Representative Window

Before constructing and comparing multiple proxy variants, we begin with a controlled and minimal setup.

We do not load the entire sequence or all windows yet.

Instead, we select **one representative window** from the validated multi-window structure defined in Notebook 02.

This ensures:

- controlled computational load,
- clean debugging and inspection,
- consistent behavior before scaling up.

We choose:

- one **medium-scale window**,
- which provides a balance between:
  - short-term noise,
  - and longer temporal variation.

From this window, we will:

- load all LiDAR frames,
- extract:
  - coordinates (x, y, z),
  - raw intensity I,
- compute geometric range:

$$R = \sqrt{x^2 + y^2 + z^2}$$

This gives us the base signals: I, R.

on which all proxy variants will be constructed.

At this stage:

- no proxy variants yet,
- no statistics yet,
- no visualization yet.

This is purely **data loading and signal preparation** for controlled analysis.

```python
import numpy as np
import pandas as pd
from pathlib import Path

candidate_roots = [
    Path("../data/semantickitti_subset/dataset/sequences/00"),
    Path("../data/semantickitti/dataset/sequences/00"),
]

DATASET_ROOT = next((p for p in candidate_roots if p.exists()),
candidate_roots[0])

print("Dataset Root:", DATASET_ROOT.resolve())
print("Exists:", DATASET_ROOT.exists())

metadata_path =
Path("../results/window_metadata/sequence00_windows.csv")
window_df = pd.read_csv(metadata_path)

print("\nLoaded Window Metadata")
print()
print("Shape:", window_df.shape)

selected_row = window_df[
    (window_df["scale"] == "medium")
].iloc[0]

start_idx = int(selected_row["start_idx"])
end_idx = int(selected_row["end_idx"])
length = int(selected_row["length"])

print("\nSelected Window")
print()
print("Window ID   :", selected_row["window_id"])
print("Scale       :", selected_row["scale"])
print("Start idx   :", start_idx)
print("End idx     :", end_idx)
print("Length      :", length)

velodyne_dir = DATASET_ROOT / "velodyne"
frame_ids = sorted([f.stem for f in velodyne_dir.glob("*.bin")])

selected_frame_ids = frame_ids[start_idx:end_idx + 1]

assert len(selected_frame_ids) == length, "Window reconstruction
failed"

print("First Frame :", selected_frame_ids[0])
print("Last Frame  :", selected_frame_ids[-1])

all_xyz = []
```

```python
all_intensity = []

for fid in selected_frame_ids:
    bin_path = velodyne_dir / f"{fid}.bin"

    scan = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)

    xyz_part = scan[:, :3]
    intensity_part = scan[:, 3]

    all_xyz.append(xyz_part)
    all_intensity.append(intensity_part)

# Concatenate
xyz = np.concatenate(all_xyz, axis=0)
intensity = np.concatenate(all_intensity, axis=0)

# Compute range
ranges = np.linalg.norm(xyz, axis=1)

print("\nLoaded Data Summary")
print()
print("Total points :", xyz.shape[0])
print("XYZ shape    :", xyz.shape)
print("Intensity    :", intensity.shape)
print("Ranges       :", ranges.shape)

assert xyz.shape[0] == intensity.shape[0] == ranges.shape[0]

print("\nSanity check passed.")
```

```
Dataset Root:
/home/twilightpriest/GitHub/reflect-aug-seg/data/semantickitti/dataset
/sequences/00
Exists: True

Loaded Window Metadata

Shape: (15, 7)

Selected Window

Window ID   : medium_0
Scale       : medium
Start idx   : 0
End idx     : 29
Length      : 30
First Frame : 000000
Last Frame  : 000029

Loaded Data Summary
```

```
Total points : 3697281
XYZ shape    : (3697281, 3)
Intensity    : (3697281,)
Ranges       : (3697281,)

Sanity check passed.
```

Construct Baseline and First Proxy (I and I · R)

With the raw signals loaded, we now begin constructing the first proxy variant.

We start with:

Raw intensity

$$\phi_0 = I$$

This serves as the baseline reference.

Range-aware proxy

$$\phi_1 = I \cdot R$$

This is the primary proxy introduced in the preliminary stage.

The goal of this step is not to compare multiple variants yet.

Instead, we:

- construct the baseline and the main proxy,
- inspect their numerical behavior,
- verify that the transformation is well-defined,
- establish a reference point for all future proxy comparisons.

We will compute:

- basic descriptive statistics,
- distribution summaries,
- and compare scale and spread.

This forms the first controlled signal comparison in the final pipeline.

```
# --- Construct signals ---
I = intensity
R = ranges

# Baseline
phi_I = I
```

```python
# Primary proxy
phi_IR = I * R

print("Signal Shapes")
print()
print("I     :", phi_I.shape)
print("I * R :", phi_IR.shape)

# Helper: describe distribution
def describe_signal(name, values):
    q = np.percentile(values, [0, 25, 50, 75, 95, 99, 100])
    print(f"\n{name}")
    print(f" min  : {q[0]:.6f}")
    print(f" p25  : {q[1]:.6f}")
    print(f" p50  : {q[2]:.6f}")
    print(f" p75  : {q[3]:.6f}")
    print(f" p95  : {q[4]:.6f}")
    print(f" p99  : {q[5]:.6f}")
    print(f" max  : {q[6]:.6f}")
    print(f" mean : {values.mean():.6f}")
    print(f" std  : {values.std():.6f}")

# Describe both signals
describe_signal("Raw Intensity (I)", phi_I)
describe_signal("Pseudo-Reflectivity (I * R)", phi_IR)
```

```
Signal Shapes

I     : (3697281,)
I * R : (3697281,)

Raw Intensity (I)
 min  : 0.000000
 p25  : 0.220000
 p50  : 0.310000
 p75  : 0.380000
 p95  : 0.550000
 p99  : 0.710000
 max  : 0.990000
 mean : 0.297202
 std  : 0.155127

Pseudo-Reflectivity (I * R)
 min  : 0.000000
 p25  : 1.607785
 p50  : 2.680925
 p75  : 4.972093
 p95  : 8.883141
 p99  : 12.038757
 max  : 72.064384
```

```
mean : 3.490287
std  : 2.890841
```

Measure Range Dependence

We now evaluate how strongly each signal depends on range.

The purpose of pseudo-reflectivity is not to remove range entirely, but to understand how the signal changes when range is explicitly incorporated.

We compute:

$$\mathrm{corr}(I, R)$$

$$\mathrm{corr}(I \cdot R, R)$$

Interpretation:

- negative or weak correlation → less range-driven.
- moderate correlation → balanced.
- strong positive correlation → range-dominated (potential overcompensation).

This step is critical for detecting whether a proxy becomes too dependent on distance rather than surface properties.

```
# Compute correlations
corr_I_R = np.corrcoef(phi_I, R)[0, 1]
corr_IR_R = np.corrcoef(phi_IR, R)[0, 1]

print("Range Correlation Analysis")
print()
print(f"corr(I, R)     : {corr_I_R:.6f}")
print(f"corr(I * R, R) : {corr_IR_R:.6f}")
```

```
Range Correlation Analysis

corr(I, R)     : -0.205967
corr(I * R, R) : 0.365598
```

Construct and Evaluate Aggressive Proxy ($I \cdot R^2$)

We now introduce a stronger range-aware transformation:

$$\phi_2 = I \cdot R^2$$

This variant applies a quadratic scaling with respect to range.

The purpose is not to improve performance, but to test whether stronger range compensation leads to:

- excessive signal amplification,

- heavy-tailed distributions,
- strong range domination,
- loss of interpretability.

This acts as a diagnostic stress test.

We will compare it against:

- raw intensity I,
- moderate proxy $I \cdot R$,

to determine whether this transformation becomes too aggressive to be useful in practice.

```
# Construct aggressive proxy
phi_IR2 = I * (R ** 2)

# Describe distribution
describe_signal("Pseudo-Reflectivity (I * R^2)", phi_IR2)

# Range correlation
corr_IR2_R = np.corrcoef(phi_IR2, R)[0, 1]

print("\nRange Correlation")
print()
print(f"corr(I * R^2, R) : {corr_IR2_R:.6f}")
```

```
Pseudo-Reflectivity (I * R^2)
 min  : 0.000000
 p25  : 9.178608
 p50  : 23.610584
 p75  : 69.515457
 p95  : 207.356888
 p99  : 417.199908
 max  : 5383.032715
 mean : 55.942745
 std  : 96.521187

Range Correlation

corr(I * R^2, R) : 0.530904
```

Rejection of Aggressive Proxy (I · R²)

We evaluated the quadratic range-aware proxy:

$$\phi_2 = I \cdot R^2$$

with the intention of testing whether stronger range compensation improves signal behavior.

Observations

The distribution statistics show:

- a very large mean and standard deviation,
- an extremely heavy upper tail (max values in the thousands),
- high percentile values (p95 and p99) indicating widespread amplification,
- a large spread relative to the mean.

The range correlation analysis shows:

$$\mathrm{corr}\left(I \cdot R^2, R\right) \approx 0.53$$

which is significantly higher than the moderate proxy $I \cdot R$.

Interpretation

This transformation leads to:

- excessive amplification of distant points,
- strong dominance of range in the signal,
- loss of balance between geometric and surface-related information,
- reduced numerical stability.

Instead of compensating for range effects, the proxy begins to **overcompensate**, making the signal increasingly driven by distance rather than underlying scene structure.

Decision

$$\phi_2 = I \cdot R^2 \rightarrow \text{Rejected}$$

This proxy is not suitable for further analysis due to:

- heavy-tailed distribution,
- high variance,
- strong range domination,
- poor interpretability.

Takeaway

This step establishes an important constraint:

> Increasing the strength of range compensation beyond a moderate level does not improve the signal. It destabilizes it.

The remainder of this notebook will focus on transformations that:

- preserve structure,
- control dynamic range,
- and maintain numerical stability.

Log-Scaled Proxy (log(1 + I · R))

We now introduce a logarithmic transformation of the primary proxy:

$$\phi_3 = \log(1 + I \cdot R)$$

Motivation:

- the $I \cdot R$ proxy expands the signal, but still exhibits a moderately heavy tail,
- logarithmic scaling compresses large values,
- reduces sensitivity to extreme range values,
- improves numerical stability,
- preserves ordering while reducing scale dominance.

This transformation is commonly used to:

- stabilize distributions,
- control dynamic range,
- improve interpretability.

We will evaluate whether this produces a more balanced signal compared to raw $I \cdot R$.

```
# Log-scaled proxy
phi_log_IR = np.log1p(phi_IR)  # log(1 + x)

# Describe distribution
describe_signal("Log-Scaled Proxy log(1 + I * R)", phi_log_IR)

# Range correlation
corr_log_IR_R = np.corrcoef(phi_log_IR, R)[0, 1]

print("\nRange Correlation")
print()
print(f"corr(log(1 + I * R), R) : {corr_log_IR_R:.6f}")
```

```
Log-Scaled Proxy log(1 + I * R)
 min  : 0.000000
 p25  : 0.958501
 p50  : 1.303164
 p75  : 1.787097
 p95  : 2.290830
 p99  : 2.567926
 max  : 4.291341
 mean : 1.305680
 std  : 0.648573

Range Correlation

corr(log(1 + I * R), R) : 0.197413
```

The logarithmic transformation:

$$\phi_3 = \log(1 + I \cdot R)$$

compresses the dynamic range of (I \cdot R) while preserving its structure.

It reduces heavy-tail effects, improves numerical stability, and lowers range dominance compared to the raw proxy.

$$\phi_3 \rightarrow \text{Retained as a strong candidate}$$

Percentile-Normalized Proxy

We now normalize the primary proxy $I \cdot R$ using percentile-based scaling.

Motivation:

- raw $I \cdot R$ has a moderate heavy tail,
- absolute values depend on scene composition,
- large values can vary across frames and windows.

We apply percentile normalization:

$$\phi_4 = \frac{I \cdot R - p_{low}}{p_{high} - p_{low}}$$

where:

- p_{low} is a lower percentile (e.g., 1%),
- p_{high} is an upper percentile (e.g., 99%).

This approach:

- clips extreme outliers,
- stabilizes the dynamic range,
- makes the signal more comparable across windows.

We will evaluate whether this improves numerical behavior without destroying useful structure.

```
# Percentile normalization
low = np.percentile(phi_IR, 1)
high = np.percentile(phi_IR, 99)

phi_norm = (phi_IR - low) / (high - low)
phi_norm = np.clip(phi_norm, 0, 1)

# Describe distribution
describe_signal("Percentile-Normalized (I * R)", phi_norm)

# Range correlation
corr_norm_R = np.corrcoef(phi_norm, R)[0, 1]
```

```
print("\nRange Correlation")
print()
print(f"corr(normalized I * R, R) : {corr_norm_R:.6f}")
```

```
Percentile-Normalized (I * R)
 min  : 0.000000
 p25  : 0.133551
 p50  : 0.222691
 p75  : 0.413007
 p95  : 0.737879
 p99  : 0.999999
 max  : 1.000000
 mean : 0.286903
 std  : 0.223450

Range Correlation

corr(normalized I * R, R) : 0.354710
```

Robust-Scaled Proxy (Median and IQR Normalization)

We now apply a robust scaling transformation to the primary proxy $I \cdot R$.

Instead of using min-max or percentile clipping, we use:

- median m,
- interquartile range (IQR):

$$\mathrm{IQR} = p\,75 - p\,25$$

The transformation is:

$$\phi_5 = \frac{I \cdot R - m}{\mathrm{IQR}}$$

Motivation:

- less sensitive to extreme values than min-max scaling,
- preserves internal structure better than hard clipping,
- centers the distribution around zero,
- scales relative to the central bulk of the data.

This produces a signal that is:

- robust to outliers,
- comparable across windows,
- numerically stable while retaining relative variation.

We will evaluate whether this provides a better balance between structure preservation and stability compared to previous variants.

```
# Robust scaling
median = np.median(phi_IR)
p25 = np.percentile(phi_IR, 25)
p75 = np.percentile(phi_IR, 75)

iqr = p75 - p25

phi_robust = (phi_IR - median) / (iqr + 1e-8)

# Describe distribution
describe_signal("Robust-Scaled (I * R)", phi_robust)

# Range correlation
corr_robust_R = np.corrcoef(phi_robust, R)[0, 1]

print("\nRange Correlation")
print()
print(f"corr(robust-scaled I * R, R) : {corr_robust_R:.6f}")
```

```
Robust-Scaled (I * R)
 min  : -0.796873
 p25  : -0.318978
 p50  : 0.000000
 p75  : 0.681022
 p95  : 1.843534
 p99  : 2.781503
 max  : 20.623402
 mean : 0.240573
 std  : 0.859268

Range Correlation

corr(robust-scaled I * R, R) : 0.365598
```

Proxy Comparison Summary

We now consolidate all evaluated proxy variants into a single comparison table.

This table summarizes:

- distribution behavior,
- numerical stability,
- range dependence,
- and overall usability.

The goal is not to declare a final winner, but to identify a **shortlist of stable and meaningful proxy candidates** for multi-window analysis in the next stage.

This table forms the final output of this notebook.

```
summary = []

def summarize(name, values, corr):
    return {
        "proxy": name,
        "mean": float(np.mean(values)),
        "std": float(np.std(values)),
        "p95": float(np.percentile(values, 95)),
        "max": float(np.max(values)),
        "corr_range": float(corr),
    }

summary.append(summarize("I", phi_I, corr_I_R))
summary.append(summarize("I * R", phi_IR, corr_IR_R))
summary.append(summarize("I * R^2", phi_IR2, corr_IR2_R))
summary.append(summarize("log(1 + I*R)", phi_log_IR, corr_log_IR_R))
summary.append(summarize("percentile_norm", phi_norm, corr_norm_R))
summary.append(summarize("robust_scaled", phi_robust, corr_robust_R))

import pandas as pd
summary_df = pd.DataFrame(summary)

print("Proxy Comparison Table")
print()
print(summary_df)
```

```
Proxy Comparison Table

             proxy       mean        std         p95           max
corr_range
0                I   0.297202   0.155127    0.550000     0.990000   -
0.205967
1            I * R   3.490287   2.890841    8.883141    72.064384
0.365598
2          I * R^2  55.942745  96.521187  207.356888  5383.032715
0.530904
3     log(1 + I*R)   1.305680   0.648573    2.290830     4.291341
0.197413
4  percentile_norm   0.286903   0.223450    0.737879     1.000000
0.354710
5    robust_scaled   0.240573   0.859268    1.843534    20.623402
0.365598
```

Conclusion

In this notebook, we constructed and evaluated a set of range-aware proxy variants derived from LiDAR intensity and geometric range.

The focus was on signal-level diagnostics:

- distribution behavior,
- numerical stability,
- and range dependence.

Key observations:

- The quadratic proxy $I \cdot R^2$ was rejected due to severe distribution explosion and strong range domination.
- The baseline proxy $I \cdot R$ remains stable and interpretable.
- The log-scaled proxy $\log(1+I \cdot R)$ provides the best balance between stability, dynamic range control, and reduced range dependence.
- Percentile normalization and robust scaling provide bounded and centered variants suitable for consistent downstream usage.

This results in a shortlist of usable proxy candidates:

$$\{I \cdot R, \log(1+I \cdot R), \text{normalized variants}\}$$

No semantic or sequence-level claims are made at this stage.

This notebook establishes a clean proxy design space and prepares the pipeline for: **multi-window temporal signal analysis in the next stage.**